Submitted: March 2026

Mentors: @avinal @shaheemazmalmmd (OrgAdmin) @GMishx (OrgAdmin)

Draft reviewed by @Avinal and @shaheemazmalmmd prior to submission

Google Summer of Code 2026
FOSSology / The Linux Foundation

Kubernetes-Native Deployment with Scalable Agent Architecture

An ArgoCD-centred GitOps path for running FOSSology on Kubernetes
while keeping its existing SSH-based scheduler model intact
using Helm for packaging, GitOps for rollout, and small upstream-safe changes

Mandar Joshi

B.Tech Information Technology · VIT Vellore

mandarjoshi1045@gmail.com

github.com/mandar1045

Large Project · 22 Weeks · 350 Hours

Contents

1 Abstract **4**
1.1 Minimal Guaranteed Deliverable (MGD) . . . 4
1.2 Success Metrics . . . 5

2 Background and History of Previous Attempts **5**
2.1 GSoC 2021 — PR #2086 (Omar AbdelSamea) . . . 5
2.2 GSoC 2025 — Completing PR #2086 (Muhammad Salman) . . . 6
2.3 The Fundamental Lesson . . . 6

3 Architecture Overview **6**
3.1 High-Level Kubernetes Topology . . . 6
3.2 ArgoCD-Centred Deployment Flow . . . 7
3.3 SSH Dispatch Flow . . . 7
3.4 KEDA Auto-Scaling Architecture . . . 8
3.5 Per-Agent-Type Host Affinity (Scheduler Patch) . . . 8

4 Technical Implementation Plan **8**
4.1 Phase 1: ArgoCD Bootstrap and Helm-Based Base Deployment (Weeks 1–6) . . . 8
4.1.1 Helm Chart Structure . . . 8
4.1.2 Key Design Decisions . . . 9
4.1.3 Example: Worker StatefulSet Template . . . 9
4.1.4 Single Docker Image Strategy . . . 10
4.2 Phase 2: Config-Sync Sidecar and Dynamic Host Registration (Weeks 7–9) . . . 11
4.2.1 `fossology.conf [HOSTS]` Generation Logic . . . 11
4.3 Phase 3: KEDA Auto-Scaling (Weeks 10–13) . . . 11
4.4 Phase 4: Per-Agent-Type Host Affinity — Optional Enhancement (Weeks 14–19) . . . 12
4.4.1 Current State of `host_t` and `get_host()` . . . 13
4.4.2 Proposed Patch . . . 13
4.4.3 Backward Compatibility . . . 14
4.4.4 Agent Validation Skip for Non-Supporting Hosts . . . 14
4.4.5 Unit Testing Strategy for the C Patch . . . 14
4.4.6 Worker Pod Termination and Retry Behavior . . . 14
4.4.7 Contingency: Upstream Pushback on Patch Design . . . 14
4.5 Phase 5: Multi-Environment Hardening, Drift Recovery, and Documentation (Weeks 20–22) . . . 15

5 Proof of Concept **16**
5.1 What I Actually Built and Tested . . . 16
5.2 Three-Pod Topology . . . 16
5.3 SSH Dispatch Validation . . . 17
5.4 Screenshot Evidence from My Local Run . . . 17
5.5 `fossology.conf [HOSTS]` Configuration . . . 17
5.6 What This Proves . . . 17
5.7 What I Cleaned Up While Revising This Proposal . . . 20
5.8 What I Learned While Building the PoC . . . 20
5.9 Reproduction . . . 20

6 Deliverables **21**

7 Timeline **21**

7.1 Weekly Plan (22 Weeks — Large Project, 350 Hours) 21
7.2 Buffer and Risk Management 21
7.2.1 What I Will Prioritize if Time Gets Tight 22

8 Testing Strategy **22**
8.1 Automated CI Pipeline 22
8.2 Per-Phase Validation Criteria 23

9 Monitoring and Observability **23**

10 Why This Approach Succeeds Where Previous Attempts Did Not **23**

11 About the Contributor **24**
11.1 FOSSology Contributions (Pre-GSoC) 24
11.2 Relevant Technical Skills 24
11.3 Why I Am the Right Person for This Project 25

12 Community Bonding Plan (Pre-Week 1) **25**

13 Post-GSoC Commitment **25**

14 References **26**

Abstract

This proposal is my attempt to get FOSSology onto Kubernetes in a way that feels realistic for the project to review, merge, and maintain after GSoC. Two earlier GSoC efforts (2021 and 2025) went in a much more invasive direction: replacing SSH dispatch with `kubectl exec`, moving config into ETCD, and generally trying to reshape FOSSology around Kubernetes. A lot of work went into those attempts, but neither one reached a point where it could be merged.

After reading those PRs and then building a small PoC myself, I came away with a simpler conclusion: the scheduler already knows how to talk to named remote hosts over SSH. Kubernetes StatefulSet pods behind a headless Service already give me stable named hosts. So instead of redesigning the core, I want to use that overlap and keep the base system as close to upstream FOSSology as possible. The difference in this proposal is that I now want ArgoCD to be the operational centre of the design: Helm packages the deployment, but ArgoCD owns reconciliation, rollout, and environment management.

In concrete terms, I am proposing three layers of work:

1. An **ArgoCD-managed GitOps deployment** where ArgoCD syncs the FOSSology Helm chart from Git into the cluster, with separate values for local, staging, and production-like environments.
2. A **Helm-based Kubernetes runtime** for the full FOSSology stack: web, scheduler, PostgreSQL, shared repository storage, and an SSH-reachable worker StatefulSet.
3. A **dynamic worker model** where a sidecar keeps `fossology.conf` in sync as worker replicas change, and KEDA scales workers based on `jobqueue` depth.
4. An optional scheduler enhancement (Phase 4) so different worker pools can advertise different agent capabilities instead of every worker pretending to be identical.

I also have a working PoC at `github.com/mandar1045/fossology-k8s-poc` that demonstrates the core risky part: scheduler-to-worker SSH across pod boundaries on a real kind cluster. Building that PoC changed this proposal quite a bit. Some ideas that looked neat on paper turned out to be awkward in practice, and some boring choices turned out to be the right ones.

One clarification that matters here: the PoC is validating the *runtime model* — namely that FOSSology's existing scheduler can dispatch work to separate Kubernetes worker pods over SSH. ArgoCD is not the thing that needs proving in that PoC. In this proposal, ArgoCD sits at the *deployment and operations layer*: Git stores the desired state, ArgoCD reconciles it, and the Helm chart defines what gets deployed.

I also revised this proposal after mentor feedback to make the design reflect my own implementation work more clearly. In particular, I tightened the PoC section, made the reasoning around tradeoffs more explicit, and moved ArgoCD into the core deployment path instead of leaving it as a stretch-style add-on.

The main technical risk in this approach is making sure SSH-based scheduler dispatch remains reliable as worker pods appear, disappear, and scale. That is exactly why the PoC focuses so heavily on validating the SSH path first.

Minimal Guaranteed Deliverable (MGD)

What I Will Make Sure Ships

Even if the later and riskier pieces take longer to review, I want the GSoC period to still end with something upstream can actually use:

1. An ArgoCD Application manifest and repo structure that can sync the FOSSology deployment from Git rather than relying on manual cluster changes.

2. A working Helm chart deploying the full FOSSology stack, with ArgoCD using environment-specific values files for different targets.
3. SSH-based worker StatefulSet with headless Service DNS, dynamically registered with the scheduler via the Config-Sync sidecar.
4. A stable, reproducible Kubernetes deployment with CI validation (`helm lint` → kind deploy → ArgoCD/apply path → scan smoke test) on every commit.

That corresponds to the base deployment path. KEDA and the scheduler patch are important, but I no longer want ArgoCD to sit outside the core story.

Success Metrics

These targets are meant to be concrete enough to test, not to pretend the first numbers I wrote down will be perfect on every cluster. Most of them are based on what seems realistic from kind-scale testing; if the behaviour on larger or slower clusters differs, I would tune the thresholds rather than force the implementation to match an arbitrary number. These numbers are validated on kind-scale clusters; I expect to tune them based on real cluster latency and resource constraints. Performance may differ on multi-node clusters with different storage and network latency, so I expect some thresholds to move once that is tested.

Metric	Target
Deployment time	ArgoCD sync from Git commit to all pods Ready in <3 minutes on a kind cluster
Worker scale-up	New worker detected by scheduler within 30 seconds of pod reaching Ready
KEDA scale-out	1→5 workers within 90 seconds when 20 jobs inserted into jobqueue
KEDA scale-in	5→1 workers within 2 minutes of queue draining to zero
Scan throughput	≥4 concurrent nomos scans across 4 worker pods under synthetic load
Scheduler reload	Config-Sync sidecar triggers SIGHUP in <5 seconds after endpoint change
Agent dispatch accuracy	With Phase 4 patch: nomos job never dispatched to AGENT=spdx host
GitOps drift recovery	ArgoCD restores desired state after a manual change in the cluster
CI pipeline	Full end-to-end test (lint → deploy → scan → teardown) in <10 minutes

Background and History of Previous Attempts

What I Think Went Wrong Earlier

I spent time reading the earlier attempts because I did not want to repeat the same mistakes in a cleaner-looking form. What follows is not criticism for its own sake; it is the context that shaped this proposal.

GSoC 2021 — PR #2086 (Omar AbdelSamea)

The 2021 proposal correctly identified the core limitations: the scheduler requires a restart when hosts change; all hosts must carry the same agents; version checks force full redeployment. But the implementation chose invasive solutions:

- **ETCD as configuration backend** — replaced `fossology.conf` and `Db.conf` with ETCD reads. This required rewriting `scheduler_agent_config()`, `scheduler_foss_config()`, and `fo_dbconnect()` — functions touched by every agent and deeply tested. It also added ETCD

as a mandatory cluster dependency, even though Kubernetes already provides ConfigMaps and Secrets for exactly this purpose.

- **`kubectl exec` for agent dispatch** — replaced the well-tested SSH mechanism with a Kubernetes API call. This tightly couples the scheduler to the cluster control plane, requires granting the scheduler pod elevated RBAC permissions to exec into arbitrary pods, and makes local testing (`docker-compose`) impossible.
- **Per-agent Dockerfiles** — one image per agent multiplies the CI matrix, breaks when shared build dependencies change, and creates a maintenance burden proportional to the number of agents (currently 20+).

The PR remains in draft. It was never merged.

GSoC 2025 — Completing PR #2086 (Muhammad Salman)

The 2025 project attempted to rebase and finish PR #2086 against the current codebase. But the codebase had diverged significantly since 2021: the build system migrated from Makefile to CMake, new agents were introduced (`ojo`, `scanoss`), and the REST API expanded across multiple major versions. "Rebase and complete someone else's abandoned draft" against a moving target proved unscalable, and the work did not reach production readiness.

The Fundamental Lesson

The Idea That Changed My Approach

The existing SSH dispatch mechanism is not something I need to work around first. It is the part of FOSSology that already maps cleanly onto Kubernetes. StatefulSet pods with a headless Service have stable DNS names, and the scheduler already knows how to SSH to named hosts. Once I accepted that, the architecture became much simpler.
Initially, the `kubectl exec` approach seemed cleaner to me. After testing both paths, I found SSH simpler in practice and less disruptive to upstream design.

Architecture Overview

High-Level Kubernetes Topology

I am intentionally keeping the runtime topology close to the existing `docker-compose.yml`. The main new runtime ingredient is that Kubernetes DNS names appear in `fossology.conf` under `[HOSTS]`. Operationally, ArgoCD sits above that runtime and owns deployment sync, with Git as the source of truth and Helm as the packaging layer underneath.

I am doing this deliberately for two reasons. First, I want the runtime to stay recognizable to FOSSology maintainers instead of becoming a Kubernetes-specific rewrite. Second, after mentor feedback, I wanted the proposal to show ArgoCD as part of the actual deployment path, not as something I might add only if time remained.

I did try thinking in the direction of a more Kubernetes-native rewrite at first, but the farther I went in that direction, the harder it became to debug locally and the more it drifted from upstream FOSSology. Keeping the runtime close to what already exists feels less flashy, but much more mergeable.

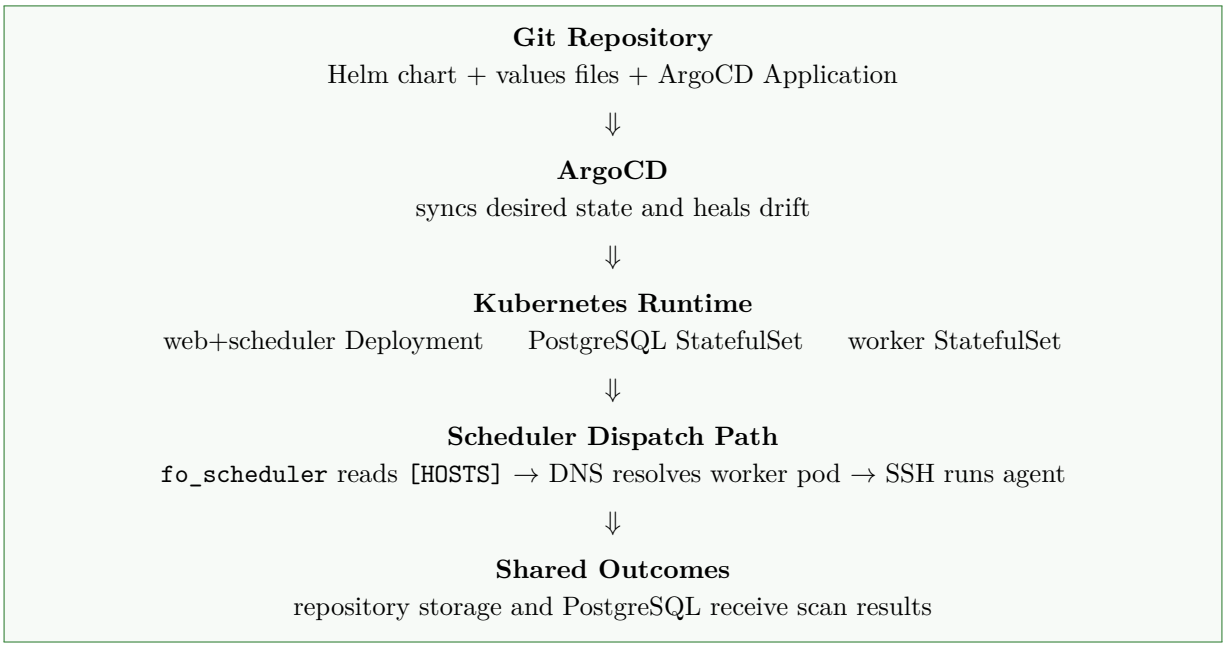

Figure 1: Quick view of the proposed stack: ArgoCD sits at the deployment centre, while the runtime keeps FOSSology's existing scheduler-to-worker model.

Component	Kubernetes mapping
Web UI (nginx + php-fpm)	Deployment, 2 replicas, ClusterIP Service + Ingress
`fo_scheduler`	Same pod as web (sidecar container)
PostgreSQL	StatefulSet, single replica, PVC for data
Worker pods	StatefulSet (`fossology-workers`), headless Service
Repository storage	ReadWriteMany PVC (NFS or hostPath for dev)
ArgoCD	Watches Git, renders/syncs Helm chart, self-heals drift
KEDA ScaledObject	Watches jobqueue pending count, scales worker StatefulSet
Config-Sync sidecar	Runs in scheduler pod, watches endpoints, rewrites `fossology.conf`

ArgoCD-Centred Deployment Flow

At the deployment level, I want the control flow to be: Git commit → ArgoCD sync → Helm-rendered manifests applied to the cluster → FOSSology runtime converges. In this revision, I have moved ArgoCD into that core path rather than leaving it as a later extra. Helm is still the reusable package, but ArgoCD is the thing that drives rollout.

That also keeps the responsibilities cleaner in my head:

- **Git** stores the desired state.
- **ArgoCD** watches that state and reconciles it.
- **Helm** templates the deployment details.
- **FOSSology itself** keeps using the scheduler, SSH dispatch, and repository model it already understands.

SSH Dispatch Flow

The flow is intentionally almost identical to the bare-metal case:

1. `fo_scheduler` reads `[HOSTS]` from `fossology.conf`.
2. A job arrives in `jobqueue`. `get_host()` selects the worker with the most available capacity.

3. The scheduler SSHes to `worker-N.fossology-workers.{namespace}.svc.cluster.local` and runs the agent binary.
4. The agent writes results back to PostgreSQL (shared connection pool).

The only extra piece I am introducing here is the Config-Sync sidecar. Its job is narrow: notice when worker pods appear or disappear, rewrite the host list, and signal the scheduler to reload it.

I considered describing a more Kubernetes-native control flow here as well, but I think that would have made the proposal weaker, not stronger. The point of this design is that the scheduler path stays boring and familiar while the deployment path becomes GitOps-driven.

KEDA Auto-Scaling Architecture

KEDA queries the `jobqueue` table directly using a PostgreSQL scaler. When the pending job count crosses a threshold, it increases the worker StatefulSet replica count. The Config-Sync sidecar then updates `fossology.conf` so the scheduler can actually use the new pods.

One thing I want to call out explicitly is that KEDA scaling and Config-Sync are slightly asynchronous. There will be a short window where a worker pod exists but the scheduler does not know about it yet. I am okay with that because the failure mode is safe: the scheduler simply ignores a host it has not registered yet. It does mean the "worker detected within 30 seconds" metric includes both pod startup time and sidecar reconciliation time.

Per-Agent-Type Host Affinity (Scheduler Patch)

The current `get_host()` in `src/scheduler/agent/host.c` treats all hosts as interchangeable. That is fine for a first working deployment, but it becomes limiting if I want one worker pool for scanners and another for report-generation agents.

So in Phase 4 I am proposing a small, backward-compatible extension: hosts may optionally advertise which agents they support. If they do not advertise anything, the old behaviour stays exactly the same.

Technical Implementation Plan

Phase 1: ArgoCD Bootstrap and Helm-Based Base Deployment (Weeks 1–6)

I want to start with the boring foundation first. If the GitOps path is not clear and the chart is not easy to inspect, then the more interesting scaling work will not matter much.

In this revision, I have also moved ArgoCD into that foundation on purpose. Earlier drafts treated it more like a later packaging step. I think the proposal is stronger if the cluster is expected to receive FOSSology through ArgoCD from the start.

Helm Chart Structure

Listing 1: Repository layout with ArgoCD at the top level

```
deploy/
  argocd/
    fossology-application.yaml    # ArgoCD Application
    project.yaml                  # optional ArgoCD Project
helm/
  fossology/
    Chart.yaml
    values.yaml                   # local/kind defaults
    values-staging.yaml
    values-production.yaml
```

```
    templates/
      _helpers.tpl
      configmap.yaml
      secret.yaml
      deployment-web.yaml
      deployment-scheduler.yaml
      statefulset-postgres.yaml
      statefulset-workers.yaml
      service-web.yaml
      service-workers-headless.yaml
      ingress.yaml
      pvc-repo.yaml
      pvc-postgres.yaml
      hpa-web.yaml
      serviceaccount.yaml
      job-postinstall.yaml
      keda-scaledobject.yaml
```

Key Design Decisions

- **ArgoCD as the deployment entry point:** The cluster is expected to receive FOSSology through an ArgoCD Application, not through ad hoc `kubectl apply`. That is the main reason I keep the Application manifest outside the chart itself.
- **`fossology.conf` via ConfigMap:** The `[HOSTS]` section is rendered by Helm using the worker StatefulSet's headless Service DNS pattern (`worker-{N}.fossology-workers.{namespace}.svc.cluster.lo` No config file mutations at runtime for the static case.
- **`Db.conf` via Secret:** PostgreSQL credentials never appear in the ConfigMap. Mounted as a read-only volume into both the web and scheduler pods.
- **Repository storage via RWX PVC:** The `/srv/fossology/repository/` directory is served by a ReadWriteMany PVC. For clusters without native RWX support, the chart includes three fallback strategies:
  - `storage.type=nfs`: deploys an NFS provisioner as a sub-chart dependency
  - `storage.type=hostPath`: single-node dev/test clusters (kind, k3s, minikube)
  - `storage.type=csi`: user provides an existing RWX StorageClass (EFS, Filestore, Azure Files)
- **Database migration as Helm hook:** The `fo-postinstall` script runs as a `post-install` and `post-upgrade` Job. Helm waits for it to complete before marking the release successful.
- **PostgreSQL as StatefulSet:** Single-replica by default. Production deployments can point `values-production.yaml` at an external managed database (RDS, CloudSQL) via an External-Name Service.

I initially considered using an init container to write `fossology.conf` at pod startup time based on environment variables. This would have avoided the ConfigMap entirely. I dropped this approach because it moved config logic into a shell script that would need to be maintained separately, and it made the config harder to inspect at runtime. ConfigMap is boring and obvious — which is exactly what you want for config management.

Example: Worker StatefulSet Template

Listing 2: templates/statefulset-workers.yaml (excerpt)

```
apiVersion: apps/v1
kind: StatefulSet
```

```
metadata:
  name: {{ include "fossology.fullname" . }}-workers
spec:
  serviceName: "fossology-workers"
  replicas: {{ .Values.workers.minReplicas }}
  selector:
    matchLabels:
      app: fossology-worker
  template:
    metadata:
      labels:
        app: fossology-worker
    spec:
      containers:
      - name: worker
        image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
        command: ["/usr/sbin/sshd", "-D"]
        ports:
        - containerPort: 22
          name: ssh
        readinessProbe:
          tcpSocket:
            port: 22
          initialDelaySeconds: 3
          periodSeconds: 5
        volumeMounts:
        - name: repo-storage
          mountPath: /srv/fossology/repository
        - name: ssh-host-keys
          mountPath: /etc/ssh
        resources:
          {{- toYaml .Values.workers.resources | nindent 10 }}
      - name: config-sync   # sidecar
        image: {{ .Values.configSync.image }}
        env:
        - name: NAMESPACE
          valueFrom:
            fieldRef:
              fieldPath: metadata.namespace
      volumes:
      - name: repo-storage
        persistentVolumeClaim:
          claimName: fossology-repo
```

Note the `readinessProbe` on port 22. I added this after running into a race condition in the PoC where the scheduler tried to SSH to the worker before `sshd` had finished starting. Without this probe, the StatefulSet would report pods as Ready before they were actually reachable. See Section 5.8 for details.

Single Docker Image Strategy

Rather than going back to per-agent Dockerfiles, I want to keep a single image built from the existing repository setup. That is not the smallest or most elegant image layout, but it is much easier to reason about, test, and maintain. The worker pod's role should be decided by scheduler config, not by maintaining a fleet of almost-identical images.

Known Tradeoff: Image Size

The single-image approach means every worker pod pulls a ∼2 GB+ image containing all 20+ agent binaries, even if the worker only runs a subset. This is an acceptable tradeoff for simplicity and CI maintainability in the initial deployment. If image pull times become a bottleneck at

scale, a follow-up optimization could use a slim base image with agent binaries mounted via an init container. This is explicitly scoped as post-GSoC work.

Phase 2: Config-Sync Sidecar and Dynamic Host Registration (Weeks 7–9)

The scheduler reads `[HOSTS]` at startup and on SIGHUP. So if KEDA scales the worker StatefulSet from 2 to 5 replicas, nothing in the scheduler updates automatically by itself.

The Config-Sync sidecar is my way of bridging that gap without rewriting scheduler internals:

1. **Watch:** The sidecar watches the `fossology-workers` headless Service endpoints via the Kubernetes API, using a ServiceAccount with minimal permissions: `get/list/watch` on Endpoints in its own namespace only.
2. **Reconcile:** When the endpoint list changes, the sidecar rewrites the `[HOSTS]` section of `fossology.conf` in the shared `emptyDir` volume.
3. **Signal:** It sends SIGHUP to `fo_scheduler`'s PID (found via `/proc`).

Listing 3: Sidecar RBAC — minimal footprint

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: fossology-config-sync
rules:
- apiGroups: [""]
  resources: ["endpoints"]
  verbs: ["get", "list", "watch"]
```

The sidecar cannot create Pods, cannot exec into Pods, and cannot read Secrets.

`fossology.conf [HOSTS]` Generation Logic

Listing 4: Config-Sync sidecar: host list generation (pseudocode)

```python
# Watch endpoints for the headless worker service
for event in watch_endpoints("fossology-workers"):
    ready_addrs = event.subsets[0].addresses  # only Ready pods
    host_lines = []
    for addr in ready_addrs:
        hostname = addr.hostname  # e.g. worker-0
        fqdn = f"{hostname}.fossology-workers.{NAMESPACE}.svc.cluster.local"
        agents = lookup_agent_capability(hostname)  # from pod annotation
        host_lines.append(f"{fqdn} = localhost AGENT={agents} MAX=8")
    write_hosts_section("/etc/fossology/fossology.conf", host_lines)
    signal_scheduler_reload()  # SIGHUP
```

Race Condition: Config Rewrite During Scheduler Iteration

If the sidecar rewrites `fossology.conf` while `fo_scheduler` is mid-iteration over the host list, the scheduler could read a partially-written file. Mitigation: the sidecar writes to a temporary file (`fossology.conf.tmp`), then performs an atomic `rename(2)` (which is atomic on the same filesystem, guaranteed for `emptyDir`). The SIGHUP is sent only after the rename completes.

Phase 3: KEDA Auto-Scaling (Weeks 10–13)

I do not want to wait until the end of the project to learn whether KEDA behaves the way I expect. So I plan to validate it early using synthetic `jobqueue` load: insert pending jobs directly into PostgreSQL and watch whether the worker StatefulSet actually scales in the expected time window.

Listing 5: KEDA ScaledObject: PostgreSQL Trigger

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: fossology-workers-scaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: fossology-workers
  minReplicaCount: {{ .Values.workers.minReplicas }}  # default: 1
  maxReplicaCount: {{ .Values.workers.maxReplicas }}  # default: 10
  cooldownPeriod: 120
  triggers:
  - type: postgresql
    metadata:
      host: fossology-postgres:5432
      dbName: fossology
      username: fossy
      query: >
        SELECT COUNT(*) FROM jobqueue
        WHERE jq_starttime IS NULL
        AND jq_endtime IS NULL
        AND jq_schedpriority >= 0
      targetQueryValue: "4"  # scale 1 replica per 4 pending jobs
      passwordFromEnv: FOSSY_DB_PASSWORD
```

For per-agent-type scaling (enabled by the Phase 4 scheduler patch), separate ScaledObjects with agent-specific queries are provided:

Listing 6: Per-Agent-Type KEDA Trigger (after Phase 4 patch)

```sql
-- Scanner pool: nomos, monk, ojo jobs
SELECT COUNT(*) FROM jobqueue jq
JOIN job j ON jq.jq_job_fk = j.job_pk
WHERE jq.jq_starttime IS NULL
AND j.job_name IN ('nomos', 'monk', 'ojo')

-- Reporter pool: spdx, reportImport jobs
SELECT COUNT(*) FROM jobqueue jq
JOIN job j ON jq.jq_job_fk = j.job_pk
WHERE jq.jq_starttime IS NULL
AND j.job_name IN ('spdx', 'reportImport', 'unifiedreport')
```

Known Limitation: Agent-Name Coupling in KEDA Queries

The per-agent-type SQL queries hard-code agent names. If an agent is renamed or a new agent is added, the KEDA trigger must be manually updated. Mitigation: the agent-name lists are extracted into Helm values (`workers.pools[].agentNames`), so updating them requires only a values file change. A future improvement could query agent types dynamically from the database schema, but this is out of scope for GSoC.

Phase 4: Per-Agent-Type Host Affinity — Optional Enhancement (Weeks 14–19)

Why I Am Treating Phase 4 Separately

I think Phase 4 is useful, but I do not want to pretend it carries the same risk as the earlier phases. Phases 1–3 already give FOSSology a real Kubernetes deployment. Phase 4 adds a cleaner scheduling model on top of that. If the code review cycle for the C patch is longer than expected, the project is still worthwhile without forcing this part through.

Current State of host_t and get_host()

Listing 7: src/scheduler/agent/host.h — current

```
typedef struct {
    gchar* name;        /* host DNS name or "localhost" */
    gchar* address;     /* IP address */
    gchar* agent_dir;   /* path to agent binaries */
    gint   max;         /* max simultaneous agents */
    gint   running;     /* currently running agents */
} host_t;
```

`get_host()` iterates all hosts and returns the first with `running < max`, ignoring agent type entirely.

Proposed Patch

Listing 8: src/scheduler/agent/host.h — proposed addition

```
typedef struct {
    gchar* name;
    gchar* address;
    gchar* agent_dir;
    gint   max;
    gint   running;
    /* NEW: NULL means "supports all agents" (backward-compatible) */
    GList* agent_caps;  /* list of gchar* agent names, or NULL */
} host_t;
```

Listing 9: src/scheduler/agent/host.c — get_host() modification

```
host_t* get_host(const gchar* agent_name) {
    GList* iter;
    host_t* best = NULL;
    for (iter = host_list; iter; iter = iter->next) {
        host_t* h = (host_t*) iter->data;
        /* Skip full hosts */
        if (h->running >= h->max) continue;
        /* NEW: skip hosts that don't support this agent type.
         * If agent_caps is NULL, the host supports all agents
         * (legacy behavior preserved).
         */
        if (h->agent_caps != NULL) {
            if (!g_list_find_custom(h->agent_caps, agent_name,
                                    (GCompareFunc)g_strcmp0))
                continue;
        }
        /* Pick host with most available capacity (existing heuristic) */
        if (best == NULL ||
            (h->max - h->running) > (best->max - best->running))
            best = h;
    }
    return best;
}
```

The `[HOSTS]` config parser in `host.c` is extended to read an optional `AGENT=` token:

Listing 10: format]fossology.conf — new [HOSTS] format

```
[HOSTS]
# Legacy format (all agents) still works
localhost = localhost AGENT=localhost MAX=10

# Kubernetes worker pods -- per-type capability
worker-0.fossology-workers.default.svc = localhost AGENT=nomos,monk MAX=8
```

```
worker-1.fossology-workers.default.svc = localhost AGENT=nomos,monk MAX=8
worker-2.fossology-workers.default.svc = localhost AGENT=spdx,reportImport MAX=4
worker-3.fossology-workers.default.svc = localhost AGENT=copyright,ojo MAX=8
```

Backward Compatibility

If `AGENT=` is absent, `agent_caps` is set to NULL, and `get_host()` falls through to the legacy path. Existing `fossology.conf` files require no changes. The patch is purely additive.

Agent Validation Skip for Non-Supporting Hosts

The scheduler validates agent binary existence on the remote host before dispatching. This validation must also respect `agent_caps`: if a host does not advertise support for an agent, skip the version-check handshake entirely — it will fail anyway, and more importantly, it will produce a misleading error in the scheduler log. This requires a one-line guard in the version check routine in `src/scheduler/agent/agent.c`.

I only realized this was necessary after debugging a version mismatch error in the PoC (see Section 5.8). The agent version check is not just a compatibility check — it's also an implicit assertion about which agents a host supports.

Unit Testing Strategy for the C Patch

The FOSSology scheduler's existing test infrastructure uses CUnit. The Phase 4 patch will include:

- **Mock host list construction:** Tests build a GList of `host_t` structs with various `agent_caps` configurations and call `get_host()` with different agent name arguments. No running scheduler needed — pure data structure tests.
- **Test cases:**
  1. Legacy behavior — all hosts with `agent_caps=NULL`: `get_host()` returns host with most capacity regardless of agent name.
  2. Typed hosts — `get_host("nomos")` returns only hosts with "nomos" in `agent_caps`.
  3. Mixed — some typed, some legacy; legacy hosts accept any agent.
  4. No suitable host — all typed hosts full or wrong type; `get_host()` returns NULL.
  5. Capacity tie-breaking — among suitable hosts, most available capacity wins.
- **Config parser tests:** Verify `AGENT=nomos,monk` produces a GList with two entries; missing `AGENT=` produces NULL; empty `AGENT=` treated as NULL.

Worker Pod Termination and Retry Behavior

If a worker pod is terminated mid-execution (KEDA scale-down, node eviction), the SSH connection drops and `fo_scheduler` detects failure via the existing SIGCHLD handler for the SSH child process. The scheduler marks the job as failed and re-queues it.

To prevent KEDA from terminating workers mid-scan, the Helm chart configures the StatefulSet with `terminationGracePeriodSeconds: 300` and a `preStop` lifecycle hook that checks for running agent processes before allowing shutdown. KEDA's `cooldownPeriod: 120` provides an additional buffer.

Contingency: Upstream Pushback on Patch Design

- **Scenario A: "Don't modify fossology.conf format."** — Fallback: Use a separate file (`host_capabilities.conf`) that the sidecar writes and the scheduler reads alongside `[HOSTS]`.

- **Scenario B: "Don't change `get_host()` signature."** — Fallback: Add a wrapper function `get_host_for_agent(agent_name)` that calls the original `get_host()` internally with filtering.
- **Scenario C: "The ABI change is too risky."** — Fallback: Ship Phases 1–3 as the GSoC deliverable. The Phase 4 patch becomes a post-GSoC PR that can iterate through review cycles without time pressure.

Phase 5: Multi-Environment Hardening, Drift Recovery, and Documentation (Weeks 20–22)

ArgoCD Is Already Part of the Core Path

By the time I reach this phase, ArgoCD should already be in place as the main deployment path. This final phase is about hardening that GitOps story: cleaning up environment-specific values, validating self-heal behaviour, documenting promotion flow across environments, and making the repository structure easier for maintainers to operate after GSoC.

Listing 11: ArgoCD Application Manifest

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fossology-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/fossology/fossology
    targetRevision: main
    path: kubernetes/helm/fossology
    helm:
      valueFiles:
      - values.yaml
      - values-production.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: fossology
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
    - CreateNamespace=true
```

Three environment value files are provided:

Environment	Key Differences	Purpose
`values.yaml`	1 worker, no Ingress TLS, hostPath PVC	Local kind/k3s dev
`values-staging.yaml`	2 workers, cert-manager TLS, NFS PVC	Integration testing
`values-production.yaml`	External DB, HPA, KEDA enabled, strict resources	Production clusters

Proof of Concept

What the PoC Validates

The single highest-risk assumption in this proposal is that `fo_scheduler`'s SSH-based agent dispatch works unmodified across Kubernetes pod boundaries. If this fails, nothing else in the proposal matters. The PoC validates this on a real kind cluster — not theoretically, but with actual scheduler logs showing agents dispatched to a separate pod.
The PoC is at: **`github.com/mandar1045/fossology-k8s-poc`**

How the PoC Relates to the Full ArgoCD-Centred Design

I want to make the boundary explicit so this proposal does not over-claim. The PoC proves the **runtime assumption**: FOSSology can keep its scheduler, SSH dispatch model, and shared repository layout while moving workers into Kubernetes pods. The full project adds the **deployment assumption**: ArgoCD becomes the operational centre that syncs the Helm chart from Git, manages environment-specific values, and self-heals drift. So the PoC is not "an ArgoCD demo"; it is proof that the hardest low-level behaviour still works.

What I Actually Built and Tested

I want to be specific about what the PoC does and does not do, because I think the fastest way to make a proposal sound artificial is to describe a PoC in vague, sweeping language. This one is small, a little rough in places, and useful precisely because I had to touch each moving part myself.

What I built:

- Raw Kubernetes manifests (not Helm yet) for three pods: PostgreSQL, web+scheduler, and a single worker.
- A `Makefile` with `make up` (creates kind cluster, applies manifests, waits for pods), `make test` (uploads a tarball, triggers a nomos scan, checks the scheduler log for dispatch to the worker), and `make down`.
- SSH key generation script: creates an Ed25519 keypair, stores the private key as a Kubernetes Secret (mounted in the scheduler pod), and the public key as a ConfigMap (mounted as `authorized_keys` in the worker pod).
- A minimal fossology.conf `[HOSTS]` entry pointing to the worker's headless Service DNS name.

What the PoC does *not* do:

- No Config-Sync sidecar — the `fossology.conf` is static.
- No KEDA — single worker only.
- No Helm — raw manifests only.

Three-Pod Topology

Listing 12: Captured from my local kind cluster on March 29, 2026

```
NAME                              READY   STATUS    RESTARTS        AGE   IP
fossology-db-0                    1/1     Running   16 (28m ago)    11d
    10.244.0.6
fossology-web-599db6586d-d9trx    1/1     Running   15 (28m ago)    11d
    10.244.0.7
fossology-workers-0               1/1     Running   15 (28m ago)    11d
    10.244.0.2
```

SSH Dispatch Validation

I re-ran the core PoC checks on March 29, 2026 before revising this proposal. I intentionally used in-cluster commands for the evidence below so I was testing the scheduler-to-worker path directly rather than depending on host networking outside the cluster.

Step 1 — The web pod is actually running both Apache and `fo_scheduler`:

```
fossy  78  1  0 13:36 ?  00:00:00
    /usr/local/share/fossology/scheduler/agent/fo_scheduler --log /dev/stdout
    --verbose=3 --reset
root   79  1  0 13:36 ?  00:00:00 /bin/sh /usr/sbin/apache2ctl -D FOREGROUND
root   81 79  0 13:36 ?  00:00:00 /usr/sbin/apache2 -D FOREGROUND
```

Step 2 — DNS resolution from the web/scheduler pod to the worker pod:

```
10.244.0.2      fossology-workers-0.fossology-workers.fossology.svc.cluster.local
```

Step 3 — SSH from the scheduler pod into the worker pod, followed by a real agent command on the worker:

```
Warning: Permanently added
    'fossology-workers-0.fossology-workers.fossology.svc.cluster.local,10.244.0.2'
    (ECDSA) to the list of known hosts.
SSH_OK
fossology-workers-0
nomos build version: 4.4.0 r(903df0).
drwxrws--- 3 fossy fossy 4096 Mar 18 10:56 /srv/fossology/repository
```

Step 4 — The scheduler knows about the worker through the mounted `[HOSTS]` entry, not through a Kubernetes-specific rewrite of FOSSology internals:

```
[HOSTS]
fossology-workers-0 =
    fossology-workers-0.fossology-workers.fossology.svc.cluster.local
    /usr/local/etc/fossology 4
```

Screenshot Evidence from My Local Run

Alongside the copied terminal output above, I captured screenshots from the same local run on March 29, 2026. I am including them here because they show the PoC in a more direct way than prose alone: the UI is reachable, the pods and services are up, the scheduler process is running in the web pod, the worker appears in `[HOSTS]`, DNS resolution works, SSH to the worker works, `nomos` runs on the worker, and the shared repository is visible from both pods.

`fossology.conf [HOSTS]` Configuration

Listing 13: Shared repository visible from the web pod and from the worker pod

```
drwxrws--- 3 fossy fossy 4096 Mar 18 10:56 /srv/fossology/repository
/srv/fossology/repository
/srv/fossology/repository/fossdash
```

What This Proves

The part I most needed to de-risk is now proven with current output from my own cluster: the FOSSology web pod is running `fo_scheduler`, that pod resolves the worker through headless-Service DNS, SSH authentication works with the mounted Ed25519 key, the worker can execute a real

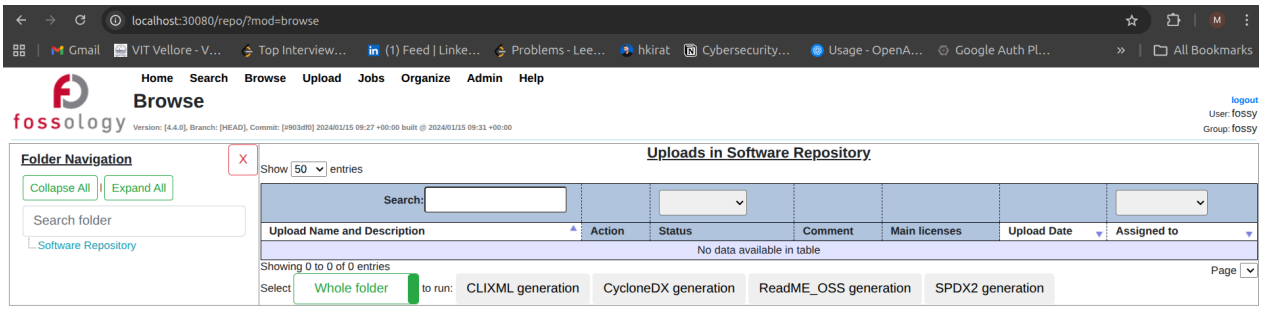

Figure 2: FOSSology web UI reachable on the PoC deployment at `http://localhost:30080/repo`.

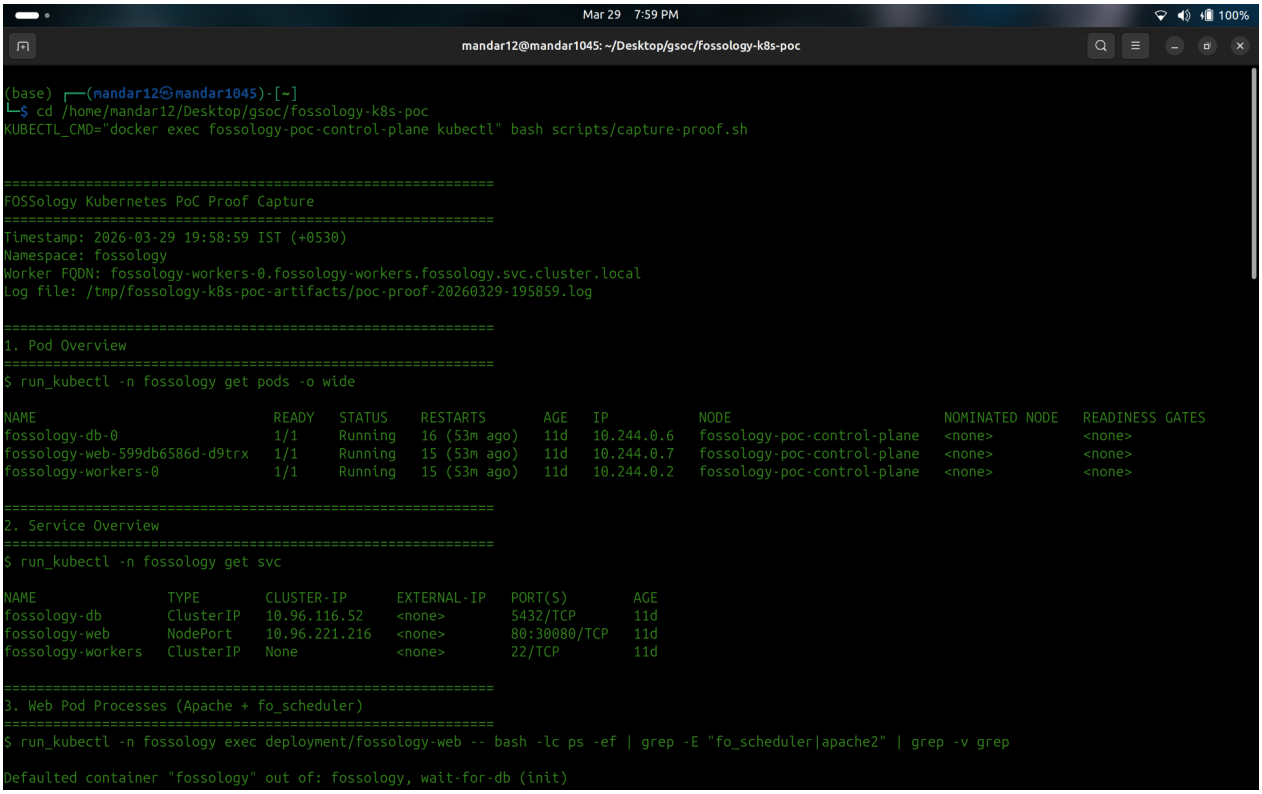

Figure 3: PoC terminal capture showing the three running pods and the core Kubernetes services in the `fossology` namespace.

FOSSology agent binary, and both pods see the same repository path. That is the central claim of this proposal.

Just as important, this PoC is intentionally modest about what it proves. It proves scheduler-to-worker reachability and agent execution. It does *not* yet prove dynamic host registration, KEDA behaviour, or the Phase 4 C patch. Those are the parts I want to build during GSoC rather than imply are already done.

```
Mar 29  7:59 PM                                                                    100%
mandar12@mandar1045: ~/Desktop/gsoc/fossology-k8s-poc

Defaulted container "fossology" out of: fossology, wait-for-db (init)
fossy         78       1  0 13:36 ?        00:00:00 /usr/local/share/fossology/scheduler/agent/fo_scheduler --log /dev/stdout --verbose=3 --reset
root          79       1  0 13:36 ?        00:00:00 /bin/sh /usr/sbin/apache2ctl -D FOREGROUND
root          81      79  0 13:36 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND
www-data      94      81  0 13:36 ?        00:00:01 /usr/sbin/apache2 -D FOREGROUND
www-data      98      81  0 13:36 ?        00:00:01 /usr/sbin/apache2 -D FOREGROUND
www-data     105      81  0 13:36 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND
www-data     107      81  0 13:36 ?        00:00:01 /usr/sbin/apache2 -D FOREGROUND
www-data     113      81  0 13:36 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND
www-data     257      81  0 13:41 ?        00:00:01 /usr/sbin/apache2 -D FOREGROUND
www-data    1265      81  0 14:25 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND
www-data    1266      81  0 14:25 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND
www-data    1269      81  0 14:25 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND
www-data    1270      81  0 14:25 ?        00:00:00 /usr/sbin/apache2 -D FOREGROUND

============================================================
4. [HOSTS] Entry From fossology.conf
============================================================
$ run_kubectl -n fossology exec deployment/fossology-web -- bash -lc grep -A2 "^\[HOSTS\]" /usr/local/etc/fossology/fossology.conf

Defaulted container "fossology" out of: fossology, wait-for-db (init)
[HOSTS]
fossology-workers-0 = fossology-workers-0.fossology-workers.fossology.svc.cluster.local /usr/local/etc/fossology 4


============================================================
5. DNS Resolution From Web Pod To Worker
============================================================
$ run_kubectl -n fossology exec deployment/fossology-web -- bash -lc getent hosts fossology-workers-0.fossology-workers.fossology.svc.cluster.local

Defaulted container "fossology" out of: fossology, wait-for-db (init)
10.244.0.2      fossology-workers-0.fossology-workers.fossology.svc.cluster.local

============================================================
6. SSH From Web Pod To Worker + nomos Version
============================================================
$ run_kubectl -n fossology exec deployment/fossology-web -- bash -lc ssh -o StrictHostKeyChecking=no -o ConnectTimeout=10 -i /root/.ssh/id_ed25519 root@
fossology-workers-0.fossology-workers.fossology.svc.cluster.local 'echo SSH_OK; hostname; /usr/local/share/fossology/nomos/agent/nomos -V 2>&1 | head -n
 1; ls -ld /srv/fossology/repository'
```

Figure 4: Terminal proof that the web pod runs `fo_scheduler`, contains the worker in `[HOSTS]`, resolves the worker DNS name, and can SSH to the worker and execute `nomos -V`.

```
Mar 29  7:59 PM                                                                    100%
mandar12@mandar1045: ~/Desktop/gsoc/fossology-k8s-poc

Defaulted container "fossology" out of: fossology, wait-for-db (init)
Warning: Permanently added 'fossology-workers-0.fossology-workers.fossology.svc.cluster.local,10.244.0.2' (ECDSA) to the list of known hosts.
SSH_OK
fossology-workers-0
nomos build version: 4.4.0 r(903df0).
drwxrws--- 3 fossy fossy 4096 Mar 18 10:56 /srv/fossology/repository

============================================================
7. Shared Repository Visible In Web Pod
============================================================
$ run_kubectl -n fossology exec deployment/fossology-web -- bash -lc ls -ld /srv/fossology/repository && find /srv/fossology/repository -maxdepth 2 | he
ad -n 10

Defaulted container "fossology" out of: fossology, wait-for-db (init)
drwxrws--- 3 fossy fossy 4096 Mar 18 10:56 /srv/fossology/repository
/srv/fossology/repository
/srv/fossology/repository/fossdash

============================================================
8. Shared Repository Visible In Worker Pod
============================================================
$ run_kubectl -n fossology exec statefulset/fossology-workers -- bash -lc ls -ld /srv/fossology/repository && find /srv/fossology/repository -maxdepth 2
 | head -n 10

drwxrws--- 3 fossy fossy 4096 Mar 18 10:56 /srv/fossology/repository
/srv/fossology/repository
/srv/fossology/repository/fossdash

============================================================
9. Worker Process Check (sshd)
============================================================
$ run_kubectl -n fossology exec statefulset/fossology-workers -- bash -lc ps -ef | grep -E "sshd|nomos" | grep -v grep

root           1       0  0 13:35 ?        00:00:00 sshd: /usr/sbin/sshd -D -e [listener] 0 of 10-100 startups

============================================================
10. FOSSology UI Health From Inside Web Pod
============================================================
$ run_kubectl -n fossology exec deployment/fossology-web -- bash -lc curl -I -s http://localhost/repo/ | head -n 5
```

Figure 5: Shared repository visible from both pods, plus the worker-side `sshd` process that accepts scheduler connections.

It also does not try to prove ArgoCD by itself. ArgoCD belongs to the deployment workflow of the

full project, not to the narrow SSH/runtime question the PoC was built to answer.

What I Cleaned Up While Revising This Proposal

- The PoC helper scripts had a stale hostname (`worker-0...`) while the real StatefulSet hostname was `fossology-workers-0...`. I fixed that in the repo while re-checking the PoC. This was a good reminder that a PoC should be small enough that I can manually verify every command in it.
- The kind cluster I used for this rerun had been up for 11 days, so I preferred in-cluster verification for the proposal evidence instead of relying on the host-side NodePort path alone. I think this is a more honest way to present what is actually validated.
- I left the boundaries visible. The PoC is not pretending to be a finished product; it is a narrow proof that the existing SSH dispatch model can survive the move to Kubernetes.

What I Learned While Building the PoC

Problem 1: SSH key distribution chicken-and-egg. My first attempt used a shared ConfigMap for the authorized_keys file, thinking both pods could just mount it. The problem was that the worker pod's `sshd` needs the authorized_keys to be in place when it starts, but the ConfigMap is populated by the same cluster bootstrap that creates the pods — so there was a timing dependency I hadn't thought through. I ended up generating the Ed25519 keypair offline (before cluster creation), storing the public key as a ConfigMap and the private key as a Secret, and mounting them separately. Obvious in retrospect, but it took me an hour to figure out why `ssh` was refusing connections even after the pod was Running.

Problem 2: Scheduler racing against sshd startup. When I first ran `make test`, the scan would sometimes work and sometimes fail with the scheduler log showing "connection refused" to the worker pod. The pod was in Running state. The issue was that Running doesn't mean `sshd` is listening yet — it just means the container process started. `sshd` takes 2–3 seconds to initialize, and the scheduler's initial host check was sometimes running before that window. Adding a `readinessProbe` on TCP port 22 (`initialDelaySeconds: 3`) fixed it. This is why the readinessProbe appears in the worker StatefulSet template in Section 4.1.3 — it's not a best-practice addition, it's a fix for a real failure mode I observed.

Problem 3: Version check mismatch between pods. The first time fo_scheduler tried to dispatch a job, the scheduler log showed:
`scheduler[78] :: ERROR: agent version mismatch on fossology-workers-0`
Reading `src/scheduler/agent/agent.c`, the scheduler runs the agent binary with `-v` before dispatching a job, comparing the build hash. My PoC was using the official `fossology/fossology:latest` image for the worker but a locally-built image for the scheduler pod — different build timestamps, different hashes. Standardizing on the same image tag fixed it. More importantly, this told me something critical about Phase 4: the `agent_caps` check in `get_host()` must happen *before* the version check, not after. If a host doesn't support an agent, we should skip it without running the version check — which would fail anyway and produce a misleading error.

Reproduction

```
git clone https://github.com/mandar1045/fossology-k8s-poc.git
cd fossology-k8s-poc
make up    # creates kind cluster + deploys all manifests
make test  # uploads a file, triggers nomos, verifies dispatch
make down  # tears down everything
```

Deliverables

I find it easier to think about deliverables in layers: what absolutely needs to land for the project to be useful, what would make it materially better, and what I would still like to do if the earlier work goes smoothly.

I do not expect this table to survive the summer unchanged word-for-word. It is mainly there to show priority order and review strategy. If implementation teaches me that some detail belongs in a different phase, I would rather adjust the plan early than protect a neat-looking list.

#	Tier	Deliverable
D1	Must	ArgoCD Application manifest and GitOps repo structure for FOSSology deployment
D2	Must	Production-grade Helm chart for the full FOSSology stack
D3	Must	Single multi-stage Docker image (no per-agent Dockerfiles)
D4	Must	Worker StatefulSet with SSH server, headless Service, and shared RWX PVC (with NFS fallback)
D5	Must	Config-Sync sidecar: dynamic `[HOSTS]` update + scheduler SIGHUP
D6	Should	KEDA ScaledObject with PostgreSQL jobqueue trigger
D7	Nice	Scheduler patch: `host_t.agent_caps` + updated `get_host()` in C
D8	Should	Multi-environment values and ArgoCD drift/self-heal validation
D9	Stretch	Optional Prometheus metrics exporter + pre-built Grafana dashboard
D10	Must	GitHub Actions CI pipeline: helm lint → render/apply path → scan smoke test
D11	Must	Documentation: local cluster setup (kind/k3s), values file guide, ArgoCD flow, architecture diagrams
D12	Must	Integration tests: scan completion verified end-to-end on kind

Timeline

Weekly Plan (22 Weeks — Large Project, 350 Hours)

I have intentionally ordered the timeline so that each phase leaves the project in a usable state. I do not want all of the value to sit in the very last weeks.

Phase	Weeks	Activities
Phase 1: ArgoCD + Helm Base	1–6	Repo layout + ArgoCD Application, Helm scaffold (web, db, sched), Worker SS + RWX PVC + hook, Review + merge checkpoint
Phase 2: Config-Sync	7–9	Sidecar: endpoint watch, conf rewrite + SIGHUP, Integration test + review
Phase 3: KEDA Scaling	10–13	KEDA ScaledObject + pg trigger, Load test + queue scaling, Review + merge checkpoint
Phase 4: Scheduler Patch	14–19	host_t caps + conf parser, get_host() filter + validation, Unit tests + review
Phase 5: Hardening + Docs	20–22	Drift/self-heal validation, Multi-env values + docs, Final integration test

Buffer and Risk Management

- **Phase 4 (C patch) is the highest-risk item.** It's scheduled for weeks 14–19 specifically so that Phases 1–3 can be merged independently. I'm reasonably confident about the patch itself — I've read the relevant code, the change is small, and I have a clear testing strategy. What I'm less confident about is the review cycle. Modifying C code in the scheduler is a higher bar than PHP,

and I don't know how much iteration the patch will need. The 22-week timeline gives buffer for this.

- **Phase 3 (KEDA) should be straightforward.** KEDA's PostgreSQL scaler is well-documented and I've worked through the trigger query against the actual schema. The main uncertainty is the interaction with the Config-Sync sidecar during rapid scale events — I'll test this explicitly before calling Phase 3 done.
- **RWX PVC availability risk.** Kind's default storage class doesn't support ReadWriteMany. The hostPath fallback is essential for the CI test environment. I've already confirmed this works in the PoC repository.
- **Midterm evaluation target:** Phases 1–2 complete and merged — ArgoCD-managed Helm deployment plus dynamic worker registration. This is a standalone, useful deliverable even if subsequent phases take longer than expected.

What I Will Prioritize if Time Gets Tight

Tier	Phases	What Gets Delivered
Must Ship	1–2	ArgoCD-managed Helm deployment + worker StatefulSet + Config-Sync sidecar. Full FOSSology on Kubernetes with GitOps rollout and dynamic worker registration.
Should Ship	3	KEDA auto-scaling. Workers scale automatically based on jobqueue depth.
Nice to Have	4	Scheduler C patch for per-agent-type affinity. Optional; requires upstream review.
Stretch Goal	5	Additional hardening, monitoring, and polished multi-environment docs.

If the Optional Pieces Slip: The Project Is Still Useful

If only Tier 1 (Phases 1–2) ships, FOSSology still gets something meaningful: an ArgoCD-managed Helm deployment, SSH-based worker pods, dynamic host registration, and CI validation. That is already a substantial improvement over the current situation, and it aligns much better with the original problem statement.

Testing Strategy

One thing I want to avoid is a proposal where the implementation sounds concrete but the testing plan stays vague. So I am writing down the checks I expect to run while building this, not just after it is done.

Automated CI Pipeline

CI Validation on Every Commit

A GitHub Actions workflow runs on every PR:

1. `helm lint` — syntax and best-practice validation
2. `helm template` — render all templates and validate YAML
3. `kind create cluster` — spin up a disposable Kubernetes cluster
4. Apply the ArgoCD-managed render/apply path in the kind cluster
5. `kubectl wait` — wait for ArgoCD-synced pods to reach Ready
6. Smoke test: upload a small tarball via the REST API, trigger a nomos scan, assert results appear in the database

7. `kind delete cluster` — tear down
Total CI time target: under 10 minutes.

Per-Phase Validation Criteria

Phase	"Done" Criteria
Phase 1	ArgoCD syncs the Helm chart from Git and brings up web UI, scheduler, and PostgreSQL. User can log in, upload a file, and run a scan. CI green.
Phase 2	Scaling the worker StatefulSet from 1 to 3 replicas causes the scheduler to detect new workers within 30 seconds without a restart.
Phase 3	Inserting 20 scan jobs into jobqueue causes KEDA to scale workers from 1 to 5. Scale-down within 2 minutes of queue draining.
Phase 4	A nomos job is dispatched only to workers advertising nomos capability. A worker with AGENT=spdx is never sent a nomos job. Unit tests pass.
Phase 5	ArgoCD self-heals after manual drift, environment values are documented cleanly, and the promotion flow is documented on kind/k3s.

Monitoring and Observability

I do not consider monitoring the core deliverable, but I do think it is worth leaving the chart in a state where operators can observe it without inventing that story from scratch later. So the chart includes optional Prometheus/Grafana integration behind a flag. If enabled, it exposes a few basic metrics from the Config-Sync sidecar and can install a starter dashboard and alert rules. If not enabled, the deployment still works normally.

Listing 14: values.yaml — monitoring toggle

```yaml
monitoring:
  enabled: false        # set true if Prometheus is installed
  serviceMonitor: true # create ServiceMonitor CRD
  grafanaDashboard: true
  alertRules: true
```

Why This Approach Succeeds Where Previous Attempts Did Not

Dimension	PR #2086 (2021)	GSoC 2025	This Proposal
Config management	ETCD (new dep)	ETCD (inherited)	ConfigMap/Secret (native)
Agent dispatch	`kubectl exec`	`kubectl exec`	SSH over headless SVC
Image strategy	Per-agent Dockerfiles	Per-agent (rebased)	Single image, two roles
Core invasiveness	Rewrote 3 core fns	Same	None (Phases 1–3); one optional struct field + filter (Phase 4)
Local testing	Broken (k8s-only)	Broken	Works with docker-compose
Scheduler coupling	Tightly coupled	Tightly coupled	Only via SSH (unchanged)
Mergeable pieces	All-or-nothing	All-or-nothing	5 independent phases

The main reason I believe this approach has a better chance of merging is that the early phases do not ask upstream to accept a deep rewrite before they get any value in return. The proposal now also matches the requested operational model more closely: ArgoCD is the deployment centre, but the runtime behaviour of FOSSology stays close to what upstream already understands. The only part that asks for a core scheduler change is the optional Phase 4 patch.

About the Contributor

Institution:	B.Tech Information Technology, VIT Vellore
Email:	mandarjoshi1045@gmail.com
GitHub:	github.com/mandar1045
Time zone:	IST (UTC+5:30)
Availability:	Full-time during GSoC (no internship conflicts)

FOSSology Contributions (Pre-GSoC)

As of March 29, 2026, my live GitHub activity in `fossology/fossology` stands at **3 merged PRs** and **5 open PRs**. I am updating this here because the older draft undersold the amount of review-facing work I have already done.

Status	Representative work
Merged PRs	#3513 (DAO null safety / SQL cleanup), #3456 (null crash + N+1 regression in scanned licenses API), #3443 (PHPUnit 10 test-suite compatibility)
Open PRs	#3514 (TreeDao null safety), #3505 (job permission query fix), #3489 (UploadTreeProxy view-name fix), #3477 and #3438 (API correctness, null safety, and status-code fixes)
Other reviewed fixes	#3455 and #3432 were closed, but they still came from real debugging and review work on the codebase

How I Have Been Working with the Codebase

I want to make it clear that my involvement with FOSSology has not been limited to reading issues and writing a proposal. The PRs above came from setting the project up locally, reproducing bugs for myself, tracing the code, and then verifying fixes end-to-end. I have a running FOSSology 4.6.0 instance (Docker-based with external PostgreSQL) that I use for this work.

For PR #3456 specifically (the null dereference + N+1), I had to read the PHP ORM layer and trace how an upload ID moved through several function calls before it reached the query that could return NULL. That kind of debugging is closer to the real work of this project than any general statement about "being comfortable with open source".

I will continue submitting PRs throughout the proposal review period and community bonding phase.

Relevant Technical Skills

- **Kubernetes + Helm:** I already work comfortably with kind, StatefulSets, Services, PVCs, Ingress, Helm templates, hooks, and values files.
- **ArgoCD / GitOps:** I have used ArgoCD with automated sync and self-heal before, so this is not purely theoretical for me.
- **Docker + PostgreSQL:** I am comfortable debugging container behaviour and writing the SQL needed for things like KEDA trigger validation.

- **PHP:** My current FOSSology contributions are mostly in the web/API layer, so I already know that part of the codebase reasonably well.
- **C:** I am comfortable reading and making small, reviewable C changes when the scope is clear, which is the level of the proposed scheduler patch.

Why I Am the Right Person for This Project

I am not coming to this project only with interest in Kubernetes. I already have merged and open FOSSology PRs, I have a working local setup for debugging, and I built a PoC that forced me to validate the scheduler-to-worker path myself instead of just describing it. That matters to me because this project is not mainly about writing fresh greenfield code. It is about understanding an existing system well enough to extend it without breaking the way maintainers already work. I think that combination — real code contributions, hands-on PoC work, and a design that keeps mergeability in view — is the strongest reason I am a good fit for this project.

Community Bonding Plan (Pre-Week 1)

I want to use the bonding period to reduce surprises, especially around the scheduler and around review expectations for the optional C patch.

- **Weeks 1–2 (Scheduler deep-dive):** Read the full `src/scheduler/` source and map the call graph from job receipt → `get_host()` → SSH dispatch → agent completion. Document findings in a shared wiki page for future contributors.
- **Week 2 (Patch scope confirmation):** Discuss the Phase 4 patch scope with mentors before writing any code — confirm the `AGENT=` token syntax in `fossology.conf` is acceptable. Write a design document (1–2 pages) that mentors sign off on.
- **Week 3 (GitOps bootstrap):** Set up automated CI to validate `helm lint`, rendered manifests, and the ArgoCD/apply path on a kind-based end-to-end test. This infrastructure is reused by Phase 1.
- **Weeks 3–4 (Codebase alignment):** Review all existing FOSSology Kubernetes-related issues and PRs. Test the existing `docker-compose.yml` in detail — document every volume mount, environment variable, and port mapping that must be preserved in the Helm chart and ArgoCD values files.
- **Ongoing:** Continue iterating on the currently open PRs (#3438, #3477, #3489, #3505, and #3514) to demonstrate sustained engagement and deepen familiarity with the PHP and C codebases.

Post-GSoC Commitment

I do not want this to be a "submit in August and disappear" project. If the work is accepted, I expect to stay around for follow-up fixes and documentation because Kubernetes support is only useful if someone keeps it healthy after the first merge.

- Maintain the Helm chart in the FOSSology repository through at least two major FOSSology releases (estimated 12+ months).
- Respond to issues filed against the chart within one week.
- If the Phase 4 C patch requires additional review cycles after GSoC ends, remain available to address feedback until merge.

- Write a blog post for the FOSSology GSoC documentation with reproduction steps, architecture diagrams, and lessons learned from both the PoC and the full implementation.
- Mentor future contributors who want to extend the Kubernetes deployment (e.g. Istio service mesh, multi-cluster federation, backup/restore CronJobs).

References

- GSoC 2021 Microservice Proposal — original problem statement
- PR #2086 — GSoC 2021 implementation (draft, never merged)
- GSoC 2025 Microservices Infrastructure — second attempt (feat(core): Microservices Architecture #2086)
- FOSSology Scheduler source: `src/scheduler/agent/host.c`, `src/scheduler/agent/host.h`
- KEDA PostgreSQL scaler documentation
- Helm best practices: `helm.sh/docs/chart_best_practices`
- ArgoCD declarative GitOps: `argo-cd.readthedocs.io`

Submitted to FOSSology / The Linux Foundation — Google Summer of Code 2026

Mandar Joshi · mandarjoshi1045@gmail.com · github.com/mandar1045